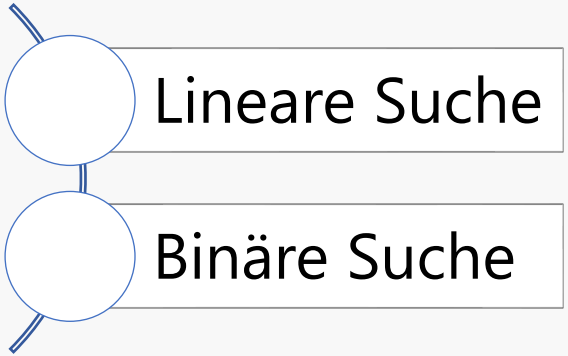
Elementare Sortieralgorithmen

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?



Was wird in diesem Foliensatz besprochen?

- Bubblesort
- Insertionsort
- Selectionsort
- Stabilität von Sortieralgorithmen

Was lernen Sie kennen?

- Einfache Sortieralgorithmen
- Unterschied zwischen stabilen und instabilen Sortieralgorithmen

Annahmen

Allgemein

- Daten (sog. Schlüssel) werden in einem Array gespeichert
- Auf den Daten ist eine Ordnungsrelation „≤“ definiert

Daten

- Alle Beispiele zuerst mit ganzen Zahlen
- Die Algorithmen funktionieren auch auf anderen Datentypen
  - Z. B. mit kleinen Änderungen bei Vergleichen

Implementierung in Methoden

- Es werden immer existierende Arrays übergeben
  - Es wird daher nicht explizit auf null überprüft
- O. B. d. A. wird immer aufsteigend sortiert

Bubblesort

Durchlauf

- Array von links nach rechts durchlaufen
- Es werden immer **zwei benachbarte** Elemente betrachtet
- Wenn sie in falscher Reihenfolge vorkommen, werden sie vertauscht

Der obige Schritt wird so oft wiederholt, bis das Array komplett sortiert ist

- Das letzte Element des vorherigen Durchlaufs muss nicht mehr beachtet werden (es befindet sich an der richtigen Position)

Implementierung

```java
private static void bubbleSort(int[] data) {
    for (int i = 0; i < data.length - 1; i++) {
        for (int j = 0; j < data.length - i - 1; j++) {
            if (data[j] > data[j + 1]) {
                exchange(data, j, j + 1);
            }
        }
    }
}

private static void exchange(int[] a, int i, int j) {
    int swap = a[i];
    a[i] = a[j];
    a[j] = swap;
}
```

Beispiel (1)

Ursprüngliches Array [73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]
Innerste Schleife (für i = 0) visualisiert

```
[2, 73, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]
[2, 19, 73, 96, 11, 5, 73, 66, 99, 1, 72, 30]
[2, 19, 73, 96, 11, 5, 73, 66, 99, 1, 72, 30]
[2, 19, 73, 11, 96, 5, 73, 66, 99, 1, 72, 30]
[2, 19, 73, 11, 5, 96, 73, 66, 99, 1, 72, 30]
[2, 19, 73, 11, 5, 73, 96, 66, 99, 1, 72, 30]
[2, 19, 73, 11, 5, 73, 66, 96, 99, 1, 72, 30]
[2, 19, 73, 11, 5, 73, 66, 96, 99, 1, 72, 30]
[2, 19, 73, 11, 5, 73, 66, 96, 1, 99, 72, 30]
[2, 19, 73, 11, 5, 73, 66, 96, 1, 72, 99, 30]
[2, 19, 73, 11, 5, 73, 66, 96, 1, 72, 30, 99]
```

```java
private static void bubbleSort(int[] data) {
    for (int i = 0; i < data.length - 1; i++) {
        for (int j = 0; j < data.length - i - 1; j++) {
            if (data[j] > data[j + 1]) {
                exchange(data, j, j + 1);
            }
        }
    }
}
```

Nach einem kompletten Durchlauf steht das Maximum (99) ganz rechts

Beispiel (2)

```
[73  2  19  96  11  5  73  66  99  1  72  30]
```

```
[2  19  73  11  5  73  66  96  1  72  30  99]
[2  19  11  5  73  66  73  1  72  30  96  99]
[2  11  5  19  66  73  1  72  30  73  96  99]
[2  5  11  19  66  1  72  30  73  73  96  99]
[2  5  11  19  1  66  30  72  73  73  96  99]
[2  5  11  1  19  30  66  72  73  73  96  99]
[2  5  1  11  19  30  66  72  73  73  96  99]
[2  1  5  11  19  30  66  72  73  73  96  99]
[1  2  5  11  19  30  66  72  73  73  96  99]
[1  2  5  11  19  30  66  72  73  73  96  99]
[1  2  5  11  19  30  66  72  73  73  96  99]
```

Obiges Array jeweils nach der inneren `for`-Schleife ausgegeben

Analyse

Zwei verschachtelte Schleifen werden immer komplett durchlaufen

- Unabhängig von der Eingabesequenz

Laufzeit ist in **$O(n^2)$**

- Anweisungen in der innersten for-Schleife haben konstante Laufzeit

Optimierung

```
private static void bubbleSort2(int[] data) {
    boolean notSorted = true;
    while (notSorted) {
        notSorted = false;
        for (int i = 0; i < data.length - 1; i++) {
            if (data[i] > data[i + 1]) {
                exchange(data, i, i + 1);
                notSorted = true;
            }
        }
    }
}
```

Abbruch, wenn in einem Durchlauf keine Elemente mehr vertauscht werden

Bubblesort optimiert – Beispiel

```
[73  2  19  96  11  5  73  66  99  1  72  30]
```

```
[2  19  73  11  5  73  66  96  1  72  30  99]
[2  19  11  5  73  66  73  1  72  30  96  99]
[2  11  5  19  66  73  1  72  30  73  96  99]
[2  5  11  19  66  1  72  30  73  73  96  99]
[2  5  11  19  1  66  30  72  73  73  96  99]
[2  5  11  1  19  30  66  72  73  73  96  99]
[2  5  1  11  19  30  66  72  73  73  96  99]
[2  1  5  11  19  30  66  72  73  73  96  99]
[1  2  5  11  19  30  66  72  73  73  96  99]
[1  2  5  11  19  30  66  72  73  73  96  99]
```

Obiges Array jeweils nach der for-Schleife ausgegeben

Bubblesort optimiert – Analyse

Schlechtester Fall (absteigend sortiertes Array)

- Beide Schleifen müssen immer durchlaufen werden
- Laufzeit ist in **$O(n^2)$**

Durchschnittlicher Fall

- Ähnlich schlechtestem Fall

Bester Fall (aufsteigend sortiertes Array)

- Optimierte Variante durchläuft Array genau einmal
- Laufzeit ist in **$O(n)$**

Bester Fall – Vergleich

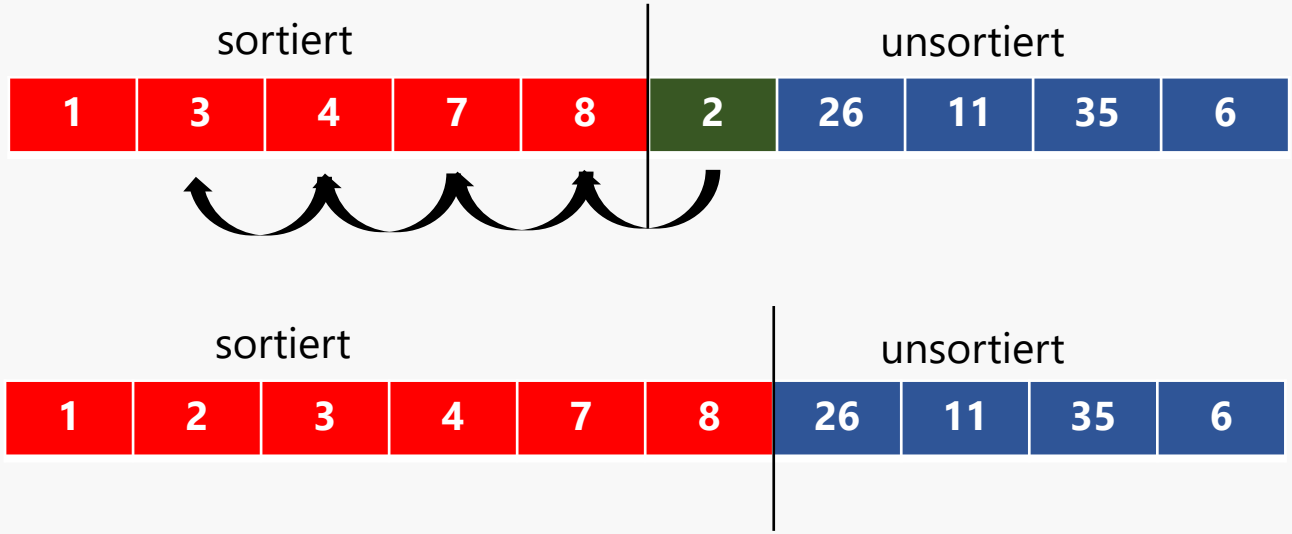
Bubblesort
Ursprüngliches Array [1 2 3 4 5 6 7 8 9 10] Ausgabe nach innerer Schleife [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10] [1 2 3 4 5 6 7 8 9 10]

Bubblesort optimiert
Ursprüngliches Array [1 2 3 4 5 6 7 8 9 10] Ausgabe nach innerer Schleife [1 2 3 4 5 6 7 8 9 10]

Insertionsort

Insertionsort nimmt aus dem unsortierten Teil der Eingabesequenz das erste Element

Fügt das Element an der richtigen Stelle in die bisher sortierte Sequenz ein



Implementierung

```
private static void insertionSort(int[] data) {
    for (int i = 1; i < data.length; i++) {
        for (int j = i; j > 0 && data[j] < data[j - 1]; j--) {
            exchange(data, j, j - 1);
        }
    }
}
```

Beispiel (1)

Ursprüngliches Array

[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]

Innerste Schleife (für i = 1) visualisiert

[**2**, **73**, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]

Innerste Schleife (für i = 2) visualisiert

[2, **19**, **73**, 96, 11, 5, 73, 66, 99, 1, 72, 30]

Innerste Schleife (für i = 4) visualisiert

[2, 19, 73, **11**, **96**, 5, 73, 66, 99, 1, 72, 30]
[2, 19, **11**, **73**, 96, 5, 73, 66, 99, 1, 72, 30]
[2, **11**, **19**, 73, 96, 5, 73, 66, 99, 1, 72, 30]

```java
private static void insertionSort(int[] data) {
    for (int i = 1; i < data.length; i++) {
        for (int j = i; j > 0 && data[j] < data[j - 1]; j--) {
            exchange(data, j, j - 1);
        }
    }
}
```

Beispiel (2)

```
[73  2  19  96  11  5  73  66  99  1  72  30]
```

```
[2  73  19  96  11  5  73  66  99  1  72  30]
[2  19  73  96  11  5  73  66  99  1  72  30]
[2  19  73  96  11  5  73  66  99  1  72  30]
[2  11  19  73  96  5  73  66  99  1  72  30]
[2  5  11  19  73  96  73  66  99  1  72  30]
[2  5  11  19  73  73  96  66  99  1  72  30]
[2  5  11  19  66  73  73  96  99  1  72  30]
[2  5  11  19  66  73  73  96  99  1  72  30]
[1  2  5  11  19  66  73  73  96  99  72  30]
[1  2  5  11  19  66  72  73  73  96  99  30]
[1  2  5  11  19  30  66  72  73  73  96  99]
```

Obiges Array jeweils nach der inneren `for`-Schleife ausgegeben

Analyse

Schlechtester Fall (absteigend sortiertes Array übergeben)

- Schleifen werden komplett durchlaufen
- Laufzeit ist in **$O(n^2)$**

Durchschnittlicher Fall

- Wie schlechtester Fall

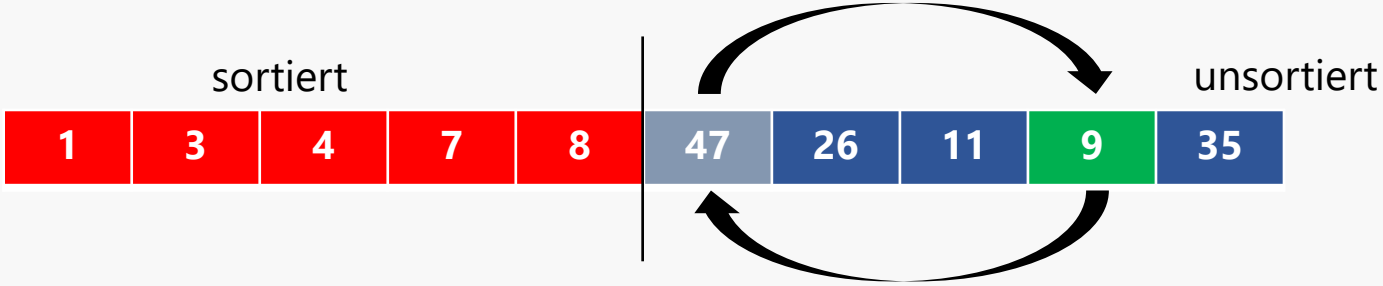
Bester Fall (aufsteigend sortiertes Array übergeben)

- Innere Schleife wird nie betreten (Bedingung wertet immer gleich auf false aus)
- Laufzeit ist in **$O(n)$**

Selectionsort

Selectionsort wählt immer das kleinste Element aus dem noch nicht sortierten Teil der Eingabesequenz

Tauscht das Element an der ersten Position nach dem sortierten Teil mit dem kleinsten Element



Implementierung

```java
private static void selectionSort(int[] data) {
    for (int i = 0; i < data.length - 1; i++) {
        int min = i;
        for (int j = i + 1; j < data.length; j++) {
            if (data[j] < data[min]) {
                min = j;
            }
        }
        exchange(data, i, min);
    }
}
```

Beispiel (1)

Ursprüngliches Array
[73, 2, 19, 96, 11, 5, 73, 66, 99, 1, 72, 30]

Innerste Schleife (für i = 0) visualisiert
Index des Minimums = 9
Array nach dem Tausch: [**1**, 2, 19, 96, 11, 5, 73, 66, 99, **73**, 72, 30]

Innerste Schleife (für i = 1) visualisiert
Index des Minimums = 1
Array nach dem Tausch: [1, **2**, 19, 96, 11, 5, 73, 66, 99, 73, 72, 30]

Innerste Schleife (für i = 2) visualisiert
Index des Minimums = 5
Array nach dem Tausch: [1, 2, **5**, 96, 11, **19**, 73, 66, 99, 73, 72, 30]

Beispiel (2)

```
[73  2  19  96  11  5  73  66  99  1  72  30]
```

```
[1  2  19  96  11  5  73  66  99  73  72  30]
[1  2  19  96  11  5  73  66  99  73  72  30]
[1  2  5  96  11  19  73  66  99  73  72  30]
[1  2  5  11  96  19  73  66  99  73  72  30]
[1  2  5  11  19  96  73  66  99  73  72  30]
[1  2  5  11  19  30  73  66  99  73  72  96]
[1  2  5  11  19  30  66  73  99  73  72  96]
[1  2  5  11  19  30  66  72  99  73  73  96]
[1  2  5  11  19  30  66  72  73  99  73  96]
[1  2  5  11  19  30  66  72  73  73  99  96]
[1  2  5  11  19  30  66  72  73  73  96  99]
```

Obiges Array jeweils nach der inneren `for`-Schleife ausgegeben

Analyse

Zwei verschachtelte Schleifen

- Werden immer komplett durchlaufen
  - Unabhängig von der Eingabesequenz
- Laufzeit ist in **$O(n^2)$**
- Bester Fall, durchschnittlicher Fall und schlechtester Fall daher gleich!

Anzahl der Aufrufe von exchange

- Linear, unabhängig von der Eingabesequenz
- Bei Insertionsort z. B. abhängig von der Eingabesequenz
  - Kann dort auch quadratisch sein

Stabilität von Sortieralgorithmen (1)

Stabiles Sortierverfahren

- Die Reihenfolge der Datensätze (komplexe Datentypen), deren Sortierschlüssel gleich sind, bleibt bewahrt

Beispiel für komplexeren Datentyp

- Personaldaten
  - Name
  - Abteilungsnummer
- Beide Informationen werden gemeinsam gespeichert
  - Implementierungsmöglichkeit wird bei der Objektorientierung noch besprochen

Stabilität von Sortieralgorithmen (2)

Beispiel (Abteilungsnummer + Name)
• Liste von Personaldaten mit Abteilungsnummer • Innerhalb der Abteilung sortiert nach Vornamen

3 **Daniel**	**4** **Maria**	**2** **Paul**	**3** **Erika**	**1** **Anton**	**3** **Sarah**

Stabilität von Sortieralgorithmen (3)

Originaldaten

3 Daniel	4 Maria	2 Paul	3 Erika	1 Anton	3 Sarah

Stabile Sortierung (nach Abteilungsnummer)

1 Anton	2 Paul	3 Daniel	3 Erika	3 Sarah	4 Maria

Nicht stabile Sortierung (nach Abteilungsnummer)

1 Anton	2 Paul	3 Erika	3 Daniel	3 Sarah	4 Maria

Stabilität – Vergleich

Sequenz vereinfacht **3 4 2 3 1 3** – Ausgaben nach inneren Schleifen

Bubblesort	Insertionsort	Selectionsort
3 4 2 3 1 3	**3 4 2 3 1 3**	**3 4 2 3 1 3**
3 2 3 1 3 4	**3 4 2 3 1 3**	**1 4 2 3 3 3**
2 3 1 3 3 4	**2 3 4 3 1 3**	**1 2 4 3 3 3**
2 1 3 3 3 4	**2 3 3 4 1 3**	**1 2 3 4 3 3**
1 2 3 3 3 4	**1 2 3 3 4 3**	**1 2 3 3 4 3**
1 2 3 3 3 4	**1 2 3 3 3 4**	**1 2 3 3 3 4**
stabil	stabil	**nicht stabil**

Elementare Sortierverfahren – Vergleich

Bubblesort

- Einfach zu implementieren und stabile Sortierung

Insertionsort

- Lineare Laufzeit bei vorsortierten Sequenzen
- Stabile Sortierung

Selectionsort

- Anzahl der Vertauschungen steigt linear an
- Besser (im Durchschnitt) als Insertionsort bzw. Bubblesort, falls Vertauschungen sehr aufwändig
- Nicht stabil

Elementare Sortierverfahren – Laufzeiten

Beispiele für Laufzeiten

- Arrays der Länge *n* mit Zufallszahlen im Bereich 0 .. *n*
- Laufzeit: Mittelwert von zehn Messungen pro Eingabegröße
- System: Intel Core i7 9750H, Windows 10, Java 14

Eingabegröße n	Bubblesort (sec)	Insertionsort (sec)	Selectionsort (sec)
10000	0,10	0,01	0,03
20000	0,43	0,05	0,09
40000	1,80	0,19	0,36
80000	7,28	0,79	1,42
160000	29,17	3,14	5,66